

BICHINHOS VIRTUAIS

Documentação técnica completa do código-fonte

linha a linha · HTML · CSS · JavaScript

Este documento explica cada seção e cada linha relevante do arquivo **bichinhos_virtuais.html**. O jogo é um arquivo HTML único com ~1350 linhas, sem dependências externas, que implementa um bichinho virtual completo com pixel art, sons 8-bit, persistência no localStorage, minigame, notificações do browser e suporte a PWA.

SUMÁRIO

- Seção 1 Estrutura HTML — O esqueleto do jogo
- Seção 2 CSS — Variáveis globais e reset
- Seção 3 CSS — Layout do aparelho (device)
- Seção 4 CSS — Tela (screen), barras e notificações
- Seção 5 CSS — Overlays: seleção, morte e recordes
- Seção 6 CSS — Animações do sprite
- Seção 7 JavaScript — Configuração dos bichinhos (PET_TYPES)
- Seção 8 JavaScript — Motor de pixel art (buildSVG)
- Seção 9 JavaScript — Paletas de cores (PAL)
- Seção 10 JavaScript — Sprites: definições por bichinho
- Seção 11 JavaScript — Classe Pet: construtor e atributos
- Seção 12 JavaScript — Classe Pet: getters e estado emocional
- Seção 13 JavaScript — Classe Pet: método tick() e decaimento
- Seção 14 JavaScript — Classe Pet: ações do usuário
- Seção 15 JavaScript — Classe Pet: serialização e fromJSON
- Seção 16 JavaScript — Storage (localStorage)
- Seção 17 JavaScript — Sistema de áudio 8-bit (Web Audio API)
- Seção 18 JavaScript — Estado global e game loop
- Seção 19 JavaScript — Funções de UI: cores, sprite, barras
- Seção 20 JavaScript — Notificações, sparkles e eventos
- Seção 21 JavaScript — Ações do jogador (doAction)
- Seção 22 JavaScript — Tela de seleção e início do jogo
- Seção 23 JavaScript — Tela de morte e recordes
- Seção 24 JavaScript — Inicialização (window.load)
- Seção 25 Fase 4 — Minigame: abertura e arena
- Seção 26 Fase 4 — Minigame: spawn, movimento e colisão
- Seção 27 Fase 4 — Minigame: resultado e fechamento
- Seção 28 Fase 4 — Partículas, dia/noite e ovo tremendo
- Seção 29 Fase 5 — Notificações do browser
- Seção 30 Fase 5 — PWA Manifest e CSS extra

1 Estrutura HTML — O esqueleto do jogo

O arquivo começa com a declaração padrão de um documento HTML5. A tag `<html lang='pt-BR'>` informa ao navegador que o idioma é português do Brasil, o que ajuda leitores de tela e motores de busca.

```
1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Bichinhos Virtuais</title>
7   <link rel="preconnect" href="https://fonts.googleapis.com">
8   <link href="https://fonts.googleapis.com/css2?family=Press+Start+2P
9     &family=Nunito:wght@400;600;700;800&display=swap" rel="stylesheet">
```

Linha 4: charset UTF-8 garante suporte a emojis e acentos brasileiros.

Linha 5: viewport com width=device-width torna o layout responsivo em celulares.

Linhas 7–8: carrega duas fontes do Google: *Press Start 2P* (visual retrô pixel art) e *Nunito* (texto legível nos botões). O atributo `rel=preconnect` abre a conexão com o servidor de fontes antecipadamente, reduzindo latência.

■ Estrutura do body — as 5 divisões principais

O body é composto por 5 blocos estruturais principais:

<code>.app-header</code>	Título e subtítulo do jogo no topo
<code>.device</code>	O aparelho físico estilo Tamagotchi com a tela e os botões
<code>.meta-bar</code>	Barra inferior com idade do bichinho e botão de recordes
<code>#selectOverlay</code>	Overlay fullscreen de escolha e nome do bichinho
<code>#recordsOverlay</code>	Overlay fullscreen com o hall da fama

■ Estrutura interna da .device

```
362 <div class="device" id="device">
363   <div class="screen" id="screen">           <!-- tela escura -->
364     <div class="screen-glow" ...></div>       <!-- brilho colorido -->
365     <div class="notif-area" ...></div>       <!-- notificações flutuantes -->
366     <div class="zzz" ...>zzz</div>          <!-- indicador de sono -->
367     <div class="sparkles" ...></div>         <!-- estrelinhas animadas -->
368     <div class="pet-area">                   <!-- sprite + nome + fase -->
369       <div class="pet-sprite" id="petSprite"></div>
370       <div class="pet-name" id="petName">---</div>
371       <div class="pet-stage-badge" ...>---</div>
372     </div>
373     <div class="status-bars" ...>           <!-- barras: fome, felicidade... -->
374       <!-- 5 bar-row, um para cada atributo -->
375     </div>
376     <div class="death-overlay hidden" ...> <!-- aparece ao morrer -->
377   </div>
378   <div class="controls">                   <!-- grid de botões de ação -->
```

Cada div com `id=` é acessada pelo JavaScript via `document.getElementById()`. Os elementos `#petSprite`, `#petName` e as barras são atualizados a cada tick do game loop.

2 CSS — Variáveis globais e reset

Antes de qualquer estilo específico, o CSS define um reset universal e um conjunto de **variáveis CSS** (custom properties) no seletor `:root`. Isso garante que mudar uma cor afete todo o jogo de uma vez só.

```
10  *{ box-sizing:border-box; margin:0; padding:0 } /* reset universal */
11  :root{
12    --bg: #f5efe6; /* fundo bege quente da página */
13    --card: #fffdf8; /* fundo dos cards/device */
14    --border:#e8ddd0; /* bordas suaves */
15    --shadow:rgba(180,150,120,.12); /* sombra dos cards */
16    --text: #3d2c1e; /* texto principal marrom-escuro */
17    --muted: #9e8878; /* texto secundário desbotado */
18    --accent:#ff8c42; /* laranja: botão principal, destaques */
19    --tick: 400ms; /* velocidade de transições */
20  }
```

box-sizing: border-box faz com que padding e border sejam incluídos na largura total do elemento, evitando surpresas de layout. **--accent: #ff8c42** é o laranja-tamagotchi usado em botões e destaques.

Além dessas, o JavaScript injeta dinamicamente duas variáveis que mudam conforme o bichinho ativo: **--pet-color** (cor principal do bichinho) e **--pet-glow** (cor do brilho da tela), via `document.documentElement.style.setProperty()`.

3 CSS — Layout do aparelho (device)

```
22 body{
23   background: var(--bg);
24   font-family: 'Nunito', sans-serif;
25   min-height: 100vh;
26   display: flex;
27   flex-direction: column; /* empilha verticalmente */
28   align-items: center; /* centraliza horizontalmente */
29   padding: 16px 12px 40px;
30 }
31 .device{
32   width: 100%;
33   max-width: 380px; /* largura máxima = tela de celular */
34   background: var(--card);
35   border: 2px solid var(--border);
36   border-radius: 32px; /* cantos bem arredondados = visual de aparelho */
37   box-shadow: 0 8px 32px var(--shadow), 0 2px 0 var(--border);
38   overflow: hidden;
39   position: relative;
40 }
```

max-width: 380px limita o aparelho ao tamanho de um celular. **border-radius: 32px** dá a forma oval característica do Tamagotchi. **overflow: hidden** faz com que os cantos arredondados recortem o conteúdo interno corretamente.

4 CSS — Tela, barras de status e notificações

■ A tela (screen)

```
60 .screen{
61   background: #1a1a2e;           /* azul-marinho escuro */
62   margin: 20px 20px 0;
63   border-radius: 16px;
64   border: 3px solid #0d0d1a;
65   box-shadow: inset 0 2px 8px rgba(0,0,0,.5); /* profundidade interna */
66   min-height: 220px;
67   display: flex; flex-direction: column;
68   align-items: center; justify-content: center;
69 }
70 .screen::before{                 /* linhas de scan (efeito CRT) */
71   content: '';
72   position: absolute; inset: 0;
73   background: repeating-linear-gradient(
74     0deg,
75     rgba(255,255,255,.02), /* listra branca semitransparente */
76     rgba(255,255,255,.02) 1px,
77     transparent 1px,
78     transparent 3px        /* repete a cada 3px */
79   );
80   z-index: 10; pointer-events: none;
81 }
```

::before cria um elemento pseudo sem conteúdo que fica sobre a tela, gerando o efeito de scanlines de monitor CRT.

pointer-events: none garante que esse overlay não interfira nos cliques.

■ Barras de status

```
110 .status-bars{
111   width:100%; padding:0 14px 12px;
112   display: grid;
113   grid-template-columns: 1fr 1fr; /* 2 colunas iguais */
114   gap: 5px;
115 }
116 .bar-track{
117   flex:1; height:5px;
118   background: #2a2a50; /* trilho escuro */
119   border-radius: 3px; overflow: hidden;
120 }
121 .bar-fill{
122   height: 100%; border-radius: 3px;
123   transition: width .5s ease; /* anima suavemente ao mudar */
124 }
```

Cada barra é composta por um **.bar-track** (o trilho cinza escuro) e um **.bar-fill** interno que tem sua *width* atualizada pelo JS a cada tick. A transição CSS de 0.5s suaviza a mudança visualmente.

■ Notificações flutuantes

```
128 .notif{
129   background: rgba(255,255,255,.12); /* vidro translúcido */
130   backdrop-filter: blur(4px);         /* desfoque do fundo */
131   border-radius: 8px;
132   animation:
133     notifIn .3s ease,           /* entra deslizando */
134     notifOut .4s ease 2.2s forwards; /* sai após 2.2s */
135 }
136 @keyframes notifIn { from{opacity:0;transform:translateY(-6px)} to{opacity:1} }
137 @keyframes notifOut { to{opacity:0;transform:translateY(-8px)} }
```

O efeito **glassmorphism** (`backdrop-filter: blur`) cria o visual de vidro fosco. A animação usa dois keyframes encadeados: *notifIn* (0–0.3s) e *notifOut* (começa em 2.2s). O JavaScript remove o elemento do DOM após 2.8s via

setTimeout.

5 CSS — Overlays: seleção, morte e recordes

```
215 .pet-select-overlay{
216   position: fixed;           /* cobre toda a tela */
217   inset: 0;                 /* atalho para top/right/bottom/left: 0 */
218   background: rgba(30,20,10,.85);
219   backdrop-filter: blur(6px);
220   z-index: 100;
221   display: flex; flex-direction: column;
222   align-items: center; justify-content: center;
223 }
224 .pet-select-overlay.hidden{ display:none; } /* toggle simples via classe */
```

position: fixed + inset: 0 faz o overlay cobrir toda a viewport independente do scroll. A visibilidade é controlada adicionando/removendo a classe **.hidden** via `classList.add('hidden')` e `classList.remove('hidden')`.

■ Botões de ação (.btn-action)

```
186 .btn-action{
187   background: var(--bg);
188   border: 2px solid var(--border);
189   border-radius: 14px;
190   display: flex; flex-direction: column;
191   align-items: center; gap: 4px;
192   position: relative; overflow: hidden; /* para a barra de cooldown */
193 }
194 .btn-action:hover:not(:disabled){
195   border-color: var(--pet-color, #6bcb77); /* cor do bichinho ativo */
196   transform: translateY(-2px);             /* sobe 2px no hover */
197 }
198 .btn-cooldown{
199   position: absolute; bottom: 0; left: 0;
200   height: 3px; background: var(--pet-color);
201   transition: width .1s linear; /* barra encolhendo em tempo real */
202 }
```

A **.btn-cooldown** é uma barra de progresso de 3px na base do botão. O JS atualiza sua *width* a cada tick com a porcentagem de cooldown restante, mostrando ao jogador quando poderá agir novamente.

6 CSS — Animações do sprite

Cinco animações CSS são definidas para os diferentes estados emocionais do bichinho. O JavaScript aplica a classe correspondente ao elemento **#petSprite** conforme o estado.

```
342 @keyframes petBounce{ /* feliz / comendo */
343     0%,100%{ transform: translateY(0) scale(1) }
344     50%    { transform: translateY(-8px) scale(1.05) }
345 }
346 @keyframes petSway{ /* dormindo */
347     0%,100%{ transform: rotate(-4deg) }
348     50%    { transform: rotate(4deg) }
349 }
350 @keyframes petPulse{ /* doente */
351     0%,100%{ transform: scale(1) }
352     50%    { transform: scale(.93) }
353 }
354 @keyframes petWiggle{ /* bravo / brincando */
355     0%,100%{ transform: rotate(0) }
356     25%    { transform: rotate(-8deg) }
357     75%    { transform: rotate(8deg) }
358 }
359 @keyframes petShake{ /* com fome */
360     0%,100%{ transform: translateX(0) }
361     25%    { transform: translateX(-4px) }
362     75%    { transform: translateX(4px) }
363 }
364 /* Classes aplicadas pelo JS: */
365 .pet-sprite.anim-bounce { animation: petBounce .7s ease-in-out infinite; }
366 .pet-sprite.anim-sway   { animation: petSway 2s ease-in-out infinite; }
367 .pet-sprite.anim-pulse  { animation: petPulse 1.5s ease-in-out infinite; }
368 .pet-sprite.anim-wiggle { animation: petWiggle .5s ease-in-out infinite; }
369 .pet-sprite.anim-shake  { animation: petShake .3s ease-in-out 3; }
```

infinite repete a animação indefinidamente enquanto a classe estiver ativa. **petShake** usa 3 em vez de *infinite* — executa apenas 3 vezes, dando um chacoalhão pontual quando o bichinho está com fome.

7 JavaScript — Configuração dos bichinhos (PET_TYPES)

O primeiro bloco JavaScript define o objeto **PET_TYPES**, um dicionário com os metadados de cada um dos 5 bichinhos. Ele é usado em toda parte: na tela de seleção, nas notificações, nas barras de status e na tela de recordes.

```
488 const PET_TYPES = {
489   florinho: { name: 'Florinho', type: 'Planta', color: '#6bcb77',
490              glow: 'rgba(107,203,119,.25)', icon: '🌱' },
491   gotinho: { name: 'Gotinho', type: 'Agua', color: '#5bc4f5',
492             glow: 'rgba(91,196,245,.25)', icon: '💧' },
493   foguinho: { name: 'Foguinho', type: 'Fogo', color: '#ff8c42',
494              glow: 'rgba(255,140,66,.25)', icon: '🔥' },
495   nuvinha: { name: 'Nuvinha', type: 'Ar', color: '#b0c8e8',
496             glow: 'rgba(176,200,232,.25)', icon: '☁️' },
497   coelhop: { name: 'Coelhop', type: 'Magico', color: '#f48fb1',
498             glow: 'rgba(244,143,177,.25)', icon: '🐇' },
499 };
```

Cada entrada tem: **name** (nome exibido), **type** (elemento), **color** (cor hexadecimal do bichinho — usada nas bordas, barras e textos), **glow** (mesma cor em rgba com transparência para o brilho da tela) e **icon** (emoji para a tela de recordes).

8 JavaScript — Motor de pixel art (buildSVG)

Esta é uma das funções mais importantes do jogo. Ela converte uma **string de mapa de pixels** em um SVG renderizável, sem precisar de arquivos de imagem externos.

```
498 function buildSVG(pixels, palette, cell=5){
499   // 'pixels' é uma string multilinha. Cada caractere = 1 pixel.
500   // '.' = transparente. Outros chars buscam a cor na 'palette'.
501   // 'cell' = tamanho de cada pixel em px (padrão 5 = amplia 5x)
502
503   const rows = pixels.trim().split('\n').map(r => r.trim());
504   // Quebra em linhas e remove espaços das bordas
505
506   const h = rows.length;
507   const w = Math.max(...rows.map(r => r.length));
508   // Altura = número de linhas, largura = maior linha
509
510   let rects = '';
511   rows.forEach((row, y) => {
512     [...row].forEach((c, x) => {
513       if(c !== '.' && palette[c]){
514         // Para cada pixel não-transparente, gera um <rect> SVG
515         rects += `<rect x="${x*cell}" y="${y*cell}"
516                   width="${cell}" height="${cell}"
517                   fill="${palette[c]}" />`;
518       }
519     });
520   });
521
522   return `<svg xmlns="http://www.w3.org/2000/svg"
523           viewBox="0 0 ${w*cell} ${h*cell}"
524           width="${w*cell}" height="${h*cell}"
525           style="image-rendering:pixelated">
526     ${rects}
527   </svg>`;
528   // image-rendering:pixelated mantém bordas nítidas ao escalar
529 }
```

Exemplo de uso: o mapa `..GGG..\n.GGGGG.` com paleta `{G:'#6bcb77'}` e `cell=5` gera um SVG de 35×10px com retângulos verdes. Isso permite criar todos os sprites diretamente em código, sem arquivos PNG.

9 JavaScript — Paletas de cores (PAL)

O objeto **PAL** contém uma paleta de cores para cada bichinho. Cada chave é uma letra usada nos mapas de pixels de buildSVG. Letras maiúsculas geralmente representam tons mais claros, minúsculas tons mais escuros.

```
506 const PAL = {
507   florinho: {
508     G: '#6bcb77', // verde principal (Green)
509     g: '#4da85a', // verde escuro (green dark)
510     D: '#3a8040', // verde muito escuro (Dark)
511     d: '#2d6b33', // verde sombra (dark shadow)
512     L: '#a8e6b3', // verde claro (Light)
513     w: '#ffffff', // branco (white) – olhos
514     R: '#e05050', // vermelho (Red) – boca/bochechas
515     S: '#c8a882', // areia (Sand) – bengala velhinho
516     X: '#778866', // verde-cinza (morto)
517     Y: '#f9e04b', // amarelo (Yellow) – comida/brilho
518   },
519   // ... (cada bichinho tem sua paleta própria)
520 };
```

O uso de letras únicas como chaves permite mapas de pixels compactos e legíveis. Por exemplo, o mapa 'GGwGwGG' no Florinho desenha o corpo verde com dois olhos brancos no centro.

■ Nos sprites de estados como 'sick' e 'angry', cores extras são passadas via *spread*: {...PAL.florinho, R: '#cc2222'} — isso sobrescreve apenas a cor *R* na paleta base.

10 JavaScript — Sprites: definições por bichinho

O objeto **SPRITES** contém todos os sprites de todos os bichinhos. Cada sprite é uma **função que retorna o SVG**, não o SVG em si — isso economiza memória, pois o SVG só é gerado quando necessário.

```
510 const SPRITES = {
511   florinho: {
512     // FASES DE VIDA (estado 'happy' usa o sprite da fase quando feliz)
513     egg: () => buildSVG('..SSS..\n.SSSsS.\nSSSSsSS...', PAL.florinho),
514     baby: () => buildSVG('..GGG..\n.GGGGG.\nGGBGBGG...', PAL.florinho),
515     adult: () => buildSVG('..dGd.dG\n..GGGGG\n.GGGGG...', PAL.florinho),
516     old: () => buildSVG('..GgG..\n.GgGgG.\nGGGwwwGG...', PAL.florinho),
517
518     // ESTADOS EMOCIONAIS
519     happy: () => buildSVG('..dGd..\n.GGGGL.\n...', PAL.florinho),
520     hungry: () => buildSVG('..dGd..\n.GGGGG.\n...', {...PAL.florinho, x:'#555577'})...,
521     sleeping: () => buildSVG('..dGd..\n.GGGGL.\n...', {...PAL.florinho, '~': '#8080aa'}...,
522     sick: () => buildSVG('..dGd..\n.GGGGG.\n...', {...PAL.florinho, x:'#555577'})...,
523     eating: () => buildSVG('..YdGdY.\n.GGGGL.\n...', PAL.florinho),
524     playing: () => buildSVG('..dGd..\n.GGGGL.\n...', PAL.florinho),
525     happy2: () => buildSVG('..YdGdY.\n.GGGGGGLG\n...', PAL.florinho),
526     angry: () => buildSVG('..dGGGd.\n.GGGGL.\n...', PAL.florinho),
527     dead: () => buildSVG('..XXx..\n.XXXXX.\n...', PAL.florinho),
528   },
529   // gotinho, foguinho, nuvinha, coelhop seguem o mesmo padrão
530 };
531
532 function getSprite(kind, state){
533   const def = SPRITES[kind];
534   if(!def) return '';
535   const fn = def[state] || def.happy; // fallback para 'happy' se estado não existir
536   return fn ? fn() : ''; // chama a função e retorna o SVG
537 }
```

A função **getSprite(kind, state)** é chamada pelo JS toda vez que precisa renderizar o bichinho. O fallback `def[state] || def.happy` garante que sempre haja um sprite, mesmo que o estado específico não exista para aquele bichinho.

11 JavaScript — Classe Pet: construtor e atributos

A classe **Pet** é o coração do jogo. Ela encapsula todo o estado do bichinho e sua lógica de comportamento. O construtor aceita um objeto de configuração *cfg*, o que permite tanto criar um novo bichinho quanto restaurar um salvo.

```
604 const STAGE_THRESHOLDS = {egg:0, baby:0.08, adult:0.5, old:2.0};
605 // Horas de jogo necessárias para cada fase:
606 // baby = 0.08h = ~5 minutos
607 // adult = 0.5h = 30 minutos
608 // old = 2.0h = 2 horas
609
610 const STAGE_LABELS = {egg:'Ovo', baby:'Filhote', adult:'Adulto', old:'Velhinho'};
611
612 const DECAY = {hunger:1.2, happiness:0.8, energy:0.6, hygiene:0.4};
613 // Taxa de decaimento por minuto de jogo:
614 // fome aumenta 1.2 por minuto, felicidade cai 0.8, etc.
615
616 class Pet {
617   constructor(cfg={}){
618     this.id = cfg.id || Date.now().toString(); // ID único
619     this.kind = cfg.kind || 'florinho'; // tipo do bichinho
620     this.name = cfg.name || PET_TYPES[this.kind]?.name || 'Bichinho';
621     this.stage = cfg.stage || 'egg'; // fase de vida
622     this.isAlive = cfg.isAlive !== undefined ? cfg.isAlive : true;
623     this.isSleeping = cfg.isSleeping || false;
624     this.isEating = false; // não persiste (reseta sempre)
625     this.isPlaying = false; // não persiste (reseta sempre)
626
627     // Atributos principais (0-100):
628     this.hunger = cfg.hunger ?? 20; // 0=satisfeito, 100=morrendo de fome
629     this.happiness = cfg.happiness ?? 80; // 0=triste, 100=muito feliz
630     this.energy = cfg.energy ?? 80; // 0=exausto, 100=descansado
631     this.hygiene = cfg.hygiene ?? 80; // 0=sujo, 100=limpinho
632     this.health = cfg.health ?? 100; // 0=morto, 100=saudável
633
634     this.ageMinutes = cfg.ageMinutes || 0; // minutos totais vividos
635     this.bornAt = cfg.bornAt || Date.now();
636     this.lastTick = cfg.lastTick || Date.now();
637
638     this.stats = cfg.stats || {feeds:0, plays:0, sleeps:0, heals:0};
639     // Contadores para o painel de recordes
640
641     this._cds = {}; // timestamps dos últimos usos de cada ação
642     this._cdTime = 4000; // cooldown em ms (4 segundos)
643   }
```

O operador `??` (nullish coalescing) usa o valor padrão apenas se `cfg.hunger` for *null* ou *undefined*, mas respeita o valor 0 — importante para atributos que podem estar zerados ao carregar um save.

12 JavaScript — Classe Pet: getters e estado emocional

Os **getters** são propriedades calculadas dinamicamente. O mais importante é **emotionalState**, que determina qual sprite e animação usar.

```
634     get ageHours() { return this.ageMinutes / 60; }
635     // Converte os minutos vividos em horas para comparar com STAGE_THRESHOLDS
636
637     get kindInfo() { return PET_TYPES[this.kind] || PET_TYPES.florinho; }
638     // Retorna o objeto de config do tipo (cor, ícone, nome, etc.)
639
640     get stageLabel(){ return STAGE_LABELS[this.stage] || this.stage; }
641     // Retorna o nome legível da fase ('Filhote', 'Adulto', etc.)
642
643     get emotionalState(){
644         if(!this.isAlive)    return 'dead';        // morreu: prioridade máxima
645         if(this.isSleeping)  return 'sleeping';    // dormindo
646         if(this.isEating)    return 'eating';      // comendo (temporário, 2s)
647         if(this.isPlaying)   return 'playing';     // brincando (temporário, 2s)
648         if(this.health < 30) return 'sick';        // saúde crítica
649         if(this.hunger > 70) return 'hungry';      // muito faminto
650         if(this.happiness > 75 && this.health > 75) return 'happy2'; // muito feliz
651         if(this.happiness < 30) return 'angry';    // muito triste/bravo
652         return 'happy';                          // estado padrão
653     }
```

A ordem das verificações em **emotionalState** é crucial — é uma cascata de prioridades. Morte supera tudo. Depois, estados temporários (comendo, brincando). Depois, estados de crise (doente, faminto). Por último, os estados positivos.

13 JavaScript — Classe Pet: método `tick()` e decaimento

`tick(deltaMs)` é chamado a cada segundo pelo game loop. Recebe o tempo decorrido em milissegundos e aplica todos os efeitos de tempo: decaimento, recuperação, alertas e possível evolução ou morte.

[illegible]

14 JavaScript — Classe Pet: ações do usuário

Cada ação verifica primeiro o cooldown e as condições (vivo, acordado, etc.), depois aplica o efeito nos atributos e agenda o reset do estado temporário.

```
696 _canAct(k){ return (Date.now() - (this._cds[k]||0)) >= this._cdTime; }
697 // Verifica se passaram 4 segundos desde o último uso da ação k
698
699 _cd(k){ this._cds[k] = Date.now(); }
700 // Registra o timestamp atual como último uso
701
702 cdLeft(k){ return Math.max(0, this._cdTime - (Date.now() - (this._cds[k]||0))); }
703 // Retorna ms restantes de cooldown (para a barra visual)
704
705 feed(){
706   if(!this._canAct('feed')) return {ok:false, msg:'Aguarde...'};
707   if(this.hunger <= 5) return {ok:false, msg:`${this.name} já está satisfeito!`};
708   this.hunger = Math.max(0, this.hunger - 30); // reduz fome em 30
709   this.happiness = Math.min(100, this.happiness + 10); // bônus de felicidade
710   this.isEating = true; // ativa sprite de comer
711   this.stats.feeds++;
712   setTimeout(() => { this.isEating = false; }, 2000); // sprite dura 2 segundos
713   this._cd('feed');
714   return {ok:true, msg:`${this.name} comeu!`};
715 }
716
717 play(){
718   if(this.energy < 15) return {ok:false, msg:`${this.name} está sem energia!`};
719   this.happiness = Math.min(100, this.happiness + 20); // +20 felicidade
720   this.energy = Math.max(0, this.energy - 12); // -12 energia (cansa)
721   this.hunger = Math.min(100, this.hunger + 5); // +5 fome (exercício)
722   this.isPlaying = true;
723   setTimeout(() => { this.isPlaying = false; }, 2000);
724   this._cd('play');
725   return {ok:true, msg:`${this.name} brincou!`};
726 }
727
728 sleep(){
729   // Botão de dormir funciona como toggle
730   if(this.isSleeping){ this.isSleeping = false; return {ok:true, msg:'Acordou!'}; }
731   if(this.energy >= 90) return {ok:false, msg:`${this.name} não está com sono!`};
732   this.isSleeping = true;
733   this.stats.sleeps++;
734   return {ok:true, msg:`${this.name} foi dormir!`};
735 }
736
737 clean(){ /* +35 higiene, +5 felicidade, cooldown */ }
738 heal(){ /* +30 saúde, +5 felicidade, cooldown */ }
```

15 JavaScript — Classe Pet: serialização e fromJSON

```
706     toJSON(){
707         // Retorna um objeto simples (serializável) com todo o estado
708         return {
709             id, kind, name, stage, isAlive, isSleeping,
710             hunger, happiness, energy, hygiene, health,
711             ageMinutes, bornAt,
712             lastTick: Date.now(), // salva o momento atual
713             stats
714         };
715     }
716
717     static fromJSON(d){
718         const p = new Pet(d); // recria o bichinho com os dados salvos
719
720         // Calcula quanto tempo passou offline (máximo 4 horas)
721         const off = Math.min(
722             Date.now() - (d.lastTick || Date.now()),
723             4 * 60 * 60 * 1000 // cap de 4 horas = 14.400.000 ms
724         );
725
726         // Se ficou mais de 5 segundos offline, aplica o decaimento
727         if(off > 5000 && p.isAlive) p.tick(off);
728
729         return p;
730     }
```

toJSON() prepara os dados para o localStorage. **fromJSON()** é o método estático que reconstrói o Pet e simula o tempo offline. O cap de 4 horas evita que o bichinho morra imediatamente se o jogador ficou muito tempo fora.

16 JavaScript — Storage (localStorage)

```
719 const SAVE_KEY = 'bichinho_v1'; // chave do save atual
720 const REC_KEY = 'bichinho_recs'; // chave dos recordes
721
722 function savePet(p){
723   try{
724     localStorage.setItem(SAVE_KEY, JSON.stringify(p.toJSON()));
725     // JSON.stringify converte o objeto em string para armazenar
726   }catch(e){} // try/catch evita crash em modo privado ou storage cheio
727 }
728
729 function loadPet(){
730   try{
731     const r = localStorage.getItem(SAVE_KEY);
732     return r ? Pet.fromJSON(JSON.parse(r)) : null;
733     // JSON.parse converte a string de volta em objeto
734     // Pet.fromJSON reconstrói a instância da classe
735   }catch(e){ return null; }
736 }
737
738 function saveRecord(p){
739   const rs = loadRecords();
740   rs.push({ name:p.name, kind:p.kind, age:Math.round(p.ageMinutes),
741           stage:p.stageLabel, ts>Date.now(), stats:p.stats });
742   rs.sort((a,b) => b.age - a.age); // ordena do mais velho para o mais novo
743   rs.splice(10); // mantém no máximo 10 recordes
744   localStorage.setItem(REC_KEY, JSON.stringify(rs));
745 }
```

17 JavaScript — Sistema de áudio 8-bit (Web Audio API)

O jogo gera sons 8-bit programaticamente usando a **Web Audio API** nativa do browser, sem nenhum arquivo de áudio. A função **beep()** cria um oscilador e um gain node.

```
729 let audioCtx = null;
730 function getAudio(){
731   // Cria o contexto apenas na primeira chamada (lazy initialization)
732   if(!audioCtx)
733     audioCtx = new (window.AudioContext || window.webkitAudioContext)();
734   return audioCtx;
735 }
736
737 function beep(freq=440, dur=0.15, type='square', vol=0.12){
738   try{
739     const ctx = getAudio();
740     const o = ctx.createOscillator(); // gerador de onda
741     const g = ctx.createGain();      // controle de volume
742
743     o.connect(g);
744     g.connect(ctx.destination);      // saída para os alto-falantes
745
746     o.type = type;                   // 'square'=8bit, 'sine'=suave
747     o.frequency.value = freq;        // frequência em Hz (440=Lá)
748
749     g.gain.setValueAtTime(vol, ctx.currentTime);
750     g.gain.exponentialRampToValueAtTime(0.001, ctx.currentTime + dur);
751     // Volume decai exponencialmente até quase zero (fade-out natural)
752
753     o.start();
754     o.stop(ctx.currentTime + dur);    // para automaticamente
755   }catch(e){}
756 }
757
758 // Sons específicos — cada um usa frequências e tipos diferentes:
759 function sfxFeed(){ beep(523,.08,'square'); setTimeout(()=>beep(659,.12),90); }
760 // Duas notas em sequência: Dó (523Hz) → Mi (659Hz)
761
762 function sfxPlay(){ beep(784,.07); setTimeout(()=>beep(880),80);
763                   setTimeout(()=>beep(1047,.15),160); }
764 // Três notas subindo: Sol → Lá → Dó (sensação de alegria)
765
766 function sfxEvolve(){ [523,659,784,1047].forEach((f,i)=>
767                  setTimeout(()=>beep(f,.18,'square',.15),i*120)); }
768 // Fanfarra de 4 notas com delay progressivo
769
770 function sfxDead(){ beep(220,.4,'sawtooth',.15);
771                   setTimeout(()=>beep(196,.5,'sawtooth',.1),350); }
772 // Onda dente de serra + notas graves = som sombrio
```

18 JavaScript — Estado global e game loop

```
750 let pet          = null;    // instância atual do Pet
751 let gameLoop     = null;    // ID do setInterval (para poder cancelar)
752 let selectedKind = 'florinho'; // bichinho selecionado na tela de escolha
753 const TICK_MS    = 1000;    // game loop roda a cada 1 segundo
754
755 let lastTickTime = Date.now(); // guarda o timestamp do tick anterior
756
757 function startLoop(){
758   if(gameLoop) clearInterval(gameLoop); // para loop anterior se existir
759   lastTickTime = Date.now();
760
761   gameLoop = setInterval(() => {
762     if(!pet || !pet.isAlive) return;    // não faz nada se sem bichinho
763
764     const now    = Date.now();
765     const delta = now - lastTickTime;    // tempo real decorrido em ms
766     lastTickTime = now;
767     // Usar delta real em vez de TICK_MS fixo corrige imprecisões do
768     // setInterval e mantém o jogo justo mesmo em abas lentas
769
770     const events = pet.tick(delta); // avança a simulação
771     handleEvents(events);           // processa notificações e morte
772     updateSprite();                 // atualiza o sprite na tela
773     updateBars();                   // atualiza as barras de status
774     updateMeta();                   // atualiza nome, fase e idade
775     updateSleepBtn();               // toggle botão dormir/acordar
776     updateCooldowns();              // atualiza barras de cooldown
777
778     // Salva a cada ~5 ticks para não escrever no storage toda vez
779     if(Math.round(pet.ageMinutes * 10) % 50 === 0) savePet(pet);
780   }, TICK_MS);
781 }
```

O uso de **delta real** (diferença entre timestamps) é uma técnica importante: o *setInterval* não é preciso — pode atrasar em abas em segundo plano. Usando o tempo real decorrido, o jogo compensa esses atrasos automaticamente.

19 JavaScript — Funções de UI: cores, sprite e barras

```
761 function setColor(color, glow){
762   // Injeta as CSS vars do bichinho ativo no elemento <html>
763   document.documentElement.style.setProperty('--pet-color', color);
764   document.documentElement.style.setProperty('--pet-glow', glow);
765   // Atualiza o brilho da tela com um gradiente radial colorido
766   document.getElementById('screenGlow').style.background =
767     `radial-gradient(ellipse at 50% 10%, ${glow} 0%, transparent 70%)`;
768 }
769
770 function updateSprite(){
771   const el = document.getElementById('petSprite');
772   const state = pet.emotionalState;
773
774   // Quando feliz, usa o sprite da fase (ovo, filhote, adulto, velhinho)
775   // em vez do sprite 'happy' genérico
776   let spriteKey = state;
777   if(state === 'happy' || state === 'happy2'){
778     if(pet.stage === 'egg' || pet.stage === 'baby' || pet.stage === 'old')
779       spriteKey = pet.stage;
780   }
781
782   el.innerHTML = getSprite(pet.kind, spriteKey) || getSprite(pet.kind, pet.stage) || ...
783   // Fallback duplo: estado → fase → string vazia
784
785   el.className = 'pet-sprite'; // reseta classes de animação
786   const animMap = {
787     happy:'anim-bounce', happy2:'anim-bounce', eating:'anim-bounce',
788     playing:'anim-wiggle', sleeping:'anim-sway', sick:'anim-pulse',
789     hungry:'anim-shake', angry:'anim-wiggle', dead:''
790   };
791   if(animMap[state]) el.classList.add(animMap[state]);
792 }
793
794 function updateBars(){
795   const set = (id, val, color) => {
796     document.getElementById(id).style.width = val + '%';
797     document.getElementById(id).style.background = color;
798   };
799   // Barra de fome é INVERTIDA: barra cheia = satisfeito (100-hunger)
800   const hc = pet.hunger>70 ? '#ef4444' : pet.hunger>40 ? '#f97316' : '#6bcb77';
801   set('barHunger', 100 - pet.hunger, hc);
802
803   // Barra de saúde: verde → laranja → vermelho conforme cai
804   const heC = pet.health<30 ? '#ef4444' : pet.health<60 ? '#f97316' : '#6bcb77';
805   set('barHealth', pet.health, heC);
806   // ... (padrão similar para happiness, energy, hygiene)
807 }
```

20 JavaScript — Notificações, sparkles e eventos

```
846 let lastNotifTime = 0;
847 function showNotif(msg, cls='', dontSpam=false){
848   if(dontSpam){
849     if(Date.now() - lastNotifTime < 3000) return; // máx 1 notif a cada 3s
850     lastNotifTime = Date.now();
851   }
852   const area = document.getElementById('notifArea');
853   const el = document.createElement('div');
854   el.className = 'notif ' + (cls||''); // classes: danger, warn, good, evolve
855   el.textContent = msg;
856   area.appendChild(el);
857   setTimeout(() => el.remove(), 2800); // remove após animação terminar
858   if(cls==='danger' || cls==='warn') sfxAlert();
859 }
860
861 function triggerSparkles(){
862   const el = document.getElementById('sparkles');
863   el.innerHTML = '';
864   el.classList.add('active');
865   const emojis = ['■', '■', '■', '■', '■'];
866   for(let i = 0; i < 6; i++){
867     const s = document.createElement('div');
868     s.className = 'sparkle';
869     s.textContent = emojis[Math.floor(Math.random() * emojis.length)];
870     s.style.left = (10 + Math.random() * 80) + '%';
871     s.style.top = (10 + Math.random() * 70) + '%';
872     s.style.animationDelay = (Math.random() * 0.4) + 's';
873     el.appendChild(s);
874   }
875   setTimeout(() => { el.classList.remove('active'); el.innerHTML=''; }, 1200);
876 }
877
878 function handleEvents(events){
879   events.forEach(ev => {
880     if(ev.type === 'death') { onDeath(); return; }
881     if(ev.type === 'evolve') { showNotif(ev.msg, 'evolve'); sfxEvolve();
882                               triggerSparkles(); return; }
883     if(ev.type === 'wakeup') { showNotif(ev.msg, 'good'); return; }
884     if(ev.type === 'sleep') { showNotif(ev.msg, 'warn'); sfxSleep(); return; }
885     if(ev.type === 'alert') showNotif(ev.msg, ev.cls||'warn', true);
886   });
887 }
```

21 JavaScript — Ações do jogador e tela de morte

```
906 function doAction(action){
907     if(!pet || !pet.isAlive) return; // guarda de segurança
908     let result;
909     if(action==='feed') { result=pet.feed(); if(result.ok){sfxFeed();triggerSparkles(...
910     if(action==='play') { result=pet.play(); if(result.ok){sfxPlay();triggerSparkles(...
911     if(action==='sleep') { result=pet.sleep(); if(result.ok) updateSleepBtn(); }
912     if(action==='clean') { result=pet.clean(); if(result.ok) sfxFeed(); }
913     if(action==='heal') { result=pet.heal(); if(result.ok){sfxHeal();triggerSparkles(...
914     if(result) showNotif(result.msg, result.ok ? 'good' : '');
915     updateZzz(); // atualiza o ícone ZZZ de sono
916     savePet(pet); // salva imediatamente após toda ação
917 }
918
919 function onDeath(){
920     clearInterval(gameLoop); // para o game loop
921     gameLoop = null;
922     saveRecord(pet); // registra no hall da fama
923     deleteSave(); // apaga o save (bichinho morreu)
924     sfxDead();
925
926     const min = Math.round(pet.ageMinutes);
927     const ageStr = min<60 ? min+'min' : Math.floor(min/60)+'h '+Math.round(min%60)+'min...
928
929     document.getElementById('deathTitle').textContent = pet.name+' NOS DEIXOU...';
930     document.getElementById('deathAge').textContent = 'Viveu por '+ageStr;
931     document.getElementById('deathSprite').innerHTML = getSprite(pet.kind, 'dead')||'';
932     document.getElementById('deathOverlay').classList.remove('hidden');
933 }
```


22 JavaScript — Tela de seleção e início do jogo

```
950 function buildSelectScreen(){
951   const container = document.getElementById('petOptions');
952   container.innerHTML = '';
953   Object.entries(PET_TYPES).forEach(([k, info]) => {
954     const div = document.createElement('div');
955     div.className = 'pet-option' + (k===selectedKind ? ' selected' : '');
956     div.style.setProperty('--oc', info.color); // cor de destaque do bichinho
957     div.innerHTML = getSprite(k, 'happy') + `<div class='pet-option-name'>
958                                     ${info.name.toUpperCase()}</div>`;
959     div.addEventListener('click', () => {
960       selectedKind = k;
961       document.querySelectorAll('.pet-option').forEach(o =>
962         o.classList.remove('selected'));
963       div.classList.add('selected');
964     });
965     container.appendChild(div);
966   });
967 }
968
969 function startGame(){
970   const rawName = document.getElementById('nameInput').value.trim();
971   const name = rawName || PET_TYPES[selectedKind]?.name || 'Bichinho';
972   // Se não digitou nome, usa o nome padrão do tipo
973
974   pet = new Pet({kind: selectedKind, name}); // cria bichinho novo
975   document.getElementById('selectOverlay').classList.add('hidden');
976   setColor(pet.kindInfo.color, pet.kindInfo.glow); // tema visual
977   updateSprite(); updateBars(); updateMeta(); updateSleepBtn(); updateZzz();
978   savePet(pet); // salva imediatamente
979   startLoop(); // inicia o game loop
980   showNotif(`${name} nasceu!`, 'good');
981   sfxEvolve(); // fanfarra de nascimento
982 }
983
984 function continueGame(){
985   pet = loadPet(); // carrega do localStorage (já aplica decaimento offline)
986   if(!pet){ showSelectScreen(true); return; }
987   setColor(pet.kindInfo.color, pet.kindInfo.glow);
988   updateSprite(); updateBars(); updateMeta(); updateSleepBtn(); updateZzz();
989   startLoop();
990   if(!pet.isAlive) onDeath(); // bichinho morreu offline
991   else showNotif(`Bem-vindo de volta, ${pet.name}!`, 'good');
992 }
```

23 JavaScript — Tela de recordes

```
1006 window.showRecords = function(){
1007   // Redefinida como window.showRecords na Fase 5 para incluir stats
1008   const records = loadRecords();
1009   const list = document.getElementById('recordsList');
1010
1011   if(!records.length){
1012     list.innerHTML = '<div class="records-empty">Nenhum recorde ainda.</div>';
1013   } else {
1014     list.innerHTML = records.map((r, i) => {
1015       const m = r.age;
1016       const txt = m<60 ? m+'min' : Math.floor(m/60)+'h '+Math.round(m%60)+'m';
1017       const info = PET_TYPES[r.kind] || PET_TYPES.florinho;
1018       const medal = ['■', '■', '■'][i] || ('#'+(i+1));
1019       const st = r.stats || {};
1020       return `<div class="record-row">
1021         ${medal} ${info.icon}
1022         <b style="color:${info.color}">${r.name}</b> (${r.stage})
1023         - Viveu ${txt}
1024         <br>■${st.feeds||0} ■${st.plays||0} ■${st.heals||0} ■${st.sleeps||0}
1025       </div>`;
1026     }).join('');
1027   }
1028   document.getElementById('recordsOverlay').classList.remove('hidden');
1029 };
```

24 JavaScript — Inicialização (window.load)

```
1032 window.addEventListener('load', () => {
1033     // 'load' dispara após todos os recursos (fontes, CSS) carregarem
1034     const saved = loadPet();
1035     if(saved){
1036         continueGame(); // retoma jogo salvo
1037     } else {
1038         showSelectScreen(true); // abre tela de seleção
1039     }
1040 });
1041
1042 window.addEventListener('beforeunload', () => {
1043     // 'beforeunload' dispara quando o usuário fecha ou navega para fora
1044     if(pet) savePet(pet); // última gravação antes de sair
1045 });
```

O evento **load** garante que o JS só rode após o DOM estar completamente montado. **beforeunload** é o 'save de emergência' — garante que o progresso seja salvo mesmo que o salvamento periódico do game loop não tenha acontecido recentemente.

25 Fase 4 — Minigame: abertura e arena

O minigame 'Pega o Item' é um modo de jogo separado que sobrepõe a tela inteira. O jogador toca/clica em itens flutuantes: frutas e joias dão pontos; cocô e raio tiram pontos. Ao final, os pontos viram felicidade para o bichinho.

```
1044 let miniActive=false, miniItems=[], miniScore=0;
1045 let miniTimer=null, miniCountdown=0, miniAnimFrame=null;
1046 // miniAnimFrame guarda o ID do requestAnimationFrame para cancelar
1047
1048 const MINI_ITEMS = [
1049   {emoji:'■', pts:2, good:true },
1050   {emoji:'■', pts:3, good:true },
1051   {emoji:'■', pts:5, good:true },
1052   {emoji:'■', pts:8, good:true }, // mais raro, mais pontos
1053   {emoji:'■', pts:-3, good:false},
1054   {emoji:'■■', pts:-2, good:false},
1055 ];
1056
1057 function openMiniGame(){
1058   // Verifica se o bichinho pode jogar
1059   if(!pet || !pet.isAlive) { showNotif('Sem bichinho!',''); return; }
1060   if(pet.isSleeping) { showNotif('Está dormindo!',''); return; }
1061   if(pet.energy < 15) { showNotif('Sem energia!','warn'); return; }
1062
1063   miniActive=true; miniScore=0; miniItems=[]; miniCountdown=15;
1064
1065   // Cria o overlay se ainda não existir
1066   let ov = document.getElementById('miniOverlay');
1067   if(!ov){ ov=document.createElement('div'); ov.id='miniOverlay';
1068     document.body.appendChild(ov); }
1069   ov.style.cssText = '...fullscreen overlay styles...';
1070   ov.innerHTML = `... arena HTML com #mArena, #mScore, #mTimer ...`;
1071
1072   // Registra eventos de clique e toque na arena
1073   const arena = document.getElementById('mArena');
1074   arena.addEventListener('click', onMiniClick);
1075   arena.addEventListener('touchstart', onMiniTouch, {passive:true});
1076   // 'passive:true' melhora performance em dispositivos móveis
1077
1078   // Countdown timer
1079   miniTimer = setInterval(() => {
1080     miniCountdown--;
1081     document.getElementById('mTimer').textContent = miniCountdown;
1082     if(miniCountdown <= 0){ clearInterval(miniTimer); endMiniGame(); }
1083   }, 1000);
1084
1085   spawnMiniItems(); // começa a spawnar itens
1086 }
```



```

1083 function spawnMiniItems(){
1084   if(!miniActive) return;
1085   const arena = document.getElementById('mArena');
1086   const W=arena.offsetWidth, H=arena.offsetHeight;
1087
1088   // Escolhe um item aleatório da lista
1089   const tpl = MINI_ITEMS[Math.floor(Math.random() * MINI_ITEMS.length)];
1090   const item = {
1091     ...tpl,
1092     id: 'mi' + Date.now() + Math.random(), // ID único para cada item
1093     x: 20 + Math.random() * (W-60), // posição X aleatória
1094     y: 20 + Math.random() * (H-60), // posição Y aleatória
1095     vx: (Math.random()-.5) * 1.6, // velocidade horizontal (-0.8 a +0.8)
1096     vy: (Math.random()-.5) * 1.6, // velocidade vertical
1097     life: 2800 + Math.random() * 2000, // tempo de vida: 2.8s a 4.8s
1098   };
1099   miniItems.push(item);
1100
1101   // Cria o elemento DOM do item
1102   const el = document.createElement('div');
1103   el.id = item.id; el.dataset.pts = item.pts; // pts no dataset para o click handler
1104   el.textContent = item.emoji;
1105   arena.appendChild(el);
1106
1107   setTimeout(()=>{ el.remove(); miniItems=miniItems.filter(i=>i.id!==item.id); }, ite...
1108   // Remove automaticamente após o tempo de vida
1109
1110   if(!miniAnimFrame) animateMiniItems(); // inicia o loop de animação se parado
1111   if(miniActive) setTimeout(spawnMiniItems, 550 + Math.random()*650); // próximo spaw...
1112 }
1113
1114 function animateMiniItems(){
1115   if(!miniActive){ miniAnimFrame=null; return; }
1116   const arena=document.getElementById('mArena');
1117   const W=arena.offsetWidth, H=arena.offsetHeight;
1118
1119   miniItems.forEach(item => {
1120     item.x += item.vx; item.y += item.vy;
1121     // Rebate nas bordas (física de bilhar)
1122     if(item.x < 5 || item.x > W-38) item.vx *= -1;
1123     if(item.y < 5 || item.y > H-38) item.vy *= -1;
1124     const el = document.getElementById(item.id);
1125     if(el){ el.style.left=item.x+'px'; el.style.top=item.y+'px'; }
1126   });
1127   miniAnimFrame = requestAnimationFrame(animateMiniItems);
1128   // requestAnimationFrame é mais eficiente que setInterval para animações
1129 }
1130
1131 function hitMiniItem(el){
1132   const pts = parseInt(el.dataset.pts) || 0;
1133   miniScore += pts;
1134   document.getElementById('mScore').textContent = Math.max(0, miniScore);
1135
1136   // Mostra label de pontos (+2, -3, etc.)
1137   const burst = document.createElement('div');
1138   burst.style.cssText = `position:absolute; left:${el.style.left}; top:${el.style.top...
1139     color:${pts>0?'#f9e04b':'#ef4444'}; animation:burstUp .6s ease forwards;`;
1140   burst.textContent = (pts>0?'+'+'') + pts;
1141   el.parentElement.appendChild(burst);
1142   setTimeout(() => burst.remove(), 700);
1143
1144   if(pts > 0){ beep(660,.07,'square'); if(pts>=5) setTimeout(()=>beep(880,.1),80); }
1145   else beep(200,.15,'sawtooth');
1146
1147   el.remove();
1148   miniItems = miniItems.filter(i => i.id !== el.id);
1149 }

```

27 Fase 4 — Minigame: resultado e fechamento

```
1152 function endMiniGame(){
1153   miniActive = false;
1154   if(miniAnimFrame){ cancelAnimationFrame(miniAnimFrame); miniAnimFrame=null; }
1155
1156   // Calcula recompensas: pontos → felicidade, energia gasta
1157   const gainHappy = Math.min(30, Math.max(0, miniScore * 2));
1158   const costEnergy = Math.min(20, 8 + Math.max(0, miniScore));
1159
1160   if(pet && pet.isAlive){
1161     pet.happiness = Math.min(100, pet.happiness + gainHappy);
1162     pet.energy    = Math.max(0, pet.energy - costEnergy);
1163     pet.hunger    = Math.min(100, pet.hunger + 6); // jogar dá fome
1164     savePet(pet); sfxPlay(); triggerSparkles();
1165   }
1166
1167   // Grade S/A/B/C/D baseada na pontuação
1168   const s    = Math.max(0, miniScore);
1169   const grade = s>=20?'S' : s>=12?'A' : s>=6?'B' : s>=0?'C' : 'D';
1170   const gc    = {S:'#f9e04b', A:'#6bcb77', B:'#5bc4f5', C:'#b0c8e8', D:'#ef4444'}[grade];
1171
1172   // Substitui o conteúdo do overlay com a tela de resultado
1173   document.getElementById('miniOverlay').innerHTML = `
1174     <div>RESULTADO</div>
1175     <div style="color:${gc}; font-size:52px;">${grade}</div> <!-- grade em destaque ...
1176     <div>■ ${s} pontos</div>
1177     <div>+${gainHappy} felicidade - ${costEnergy} energia</div>
1178     <button onclick="closeMiniGame(false)">CONTINUAR</button>
1179     <button onclick="openMiniGame()">JOGAR DE NOVO</button>
1180   `;
1181 }
1182
1183 function closeMiniGame(abrupt=false){
1184   miniActive=false; clearInterval(miniTimer);
1185   if(miniAnimFrame){ cancelAnimationFrame(miniAnimFrame); miniAnimFrame=null; }
1186   document.getElementById('miniOverlay').style.display='none';
1187   if(abrupt && pet) pet.isPlaying = false; // limpa estado se saiu no meio
1188 }
```

[illegible]

29 Fase 5 — Notificações do browser

```
1230 let notifPerm = 'default', lastBNotif = 0;
1231
1232 async function requestNotifPermission(){
1233   if(!('Notification' in window)) return; // browser não suporta
1234   if(Notification.permission === 'granted'){ notifPerm='granted'; return; }
1235   if(Notification.permission !== 'denied')
1236     notifPerm = await Notification.requestPermission();
1237   // requestPermission() mostra o popup nativo do browser
1238 }
1239
1240 function sendBrowserNotif(title, body){
1241   if(notifPerm !== 'granted') return; // sem permissão: não envia
1242   if(document.visibilityState === 'visible') return; // aba ativa: desnecessário
1243   if(Date.now() - lastBNotif < 5*60*1000) return; // máx 1 notif a cada 5 min
1244   lastBNotif = Date.now();
1245   try{ new Notification(title, {body}); }catch(e){}
1246 }
1247
1248 // Verifica a cada 3 minutos se deve notificar
1249 setInterval(() => {
1250   if(!pet || !pet.isAlive) return;
1251   if(document.visibilityState === 'visible') return; // não notifica se ativo
1252   if(pet.hunger > 75)
1253     sendBrowserNotif(pet.name+' está com fome! ■', 'Volte e alimente!');
1254   else if(pet.health < 30)
1255     sendBrowserNotif(pet.name+' está doente! ■', 'Remédio urgente!');
1256   else if(pet.happiness < 20)
1257     sendBrowserNotif(pet.name+' está triste! ■', 'Brinque com ele!');
1258 }, 3*60*1000);
1259
1260 // Pede permissão na primeira interação do usuário
1261 document.addEventListener('click', () => requestNotifPermission(), {once:true});
1262 // 'once:true' remove o listener após disparar uma vez
```

[illegible]